Monte Carlo (MC) Methods

Genya Ishigaki

Impracticability of Dynamic Programming

- DP Drawbacks
  - Their assumption of a perfect model
  - Their great computational expense
- What we want is an algorithm that works
  - Without a perfect model (access to the structure of the environment)
  - Without huge iterations over state/action spaces

Monte Carlo Methods

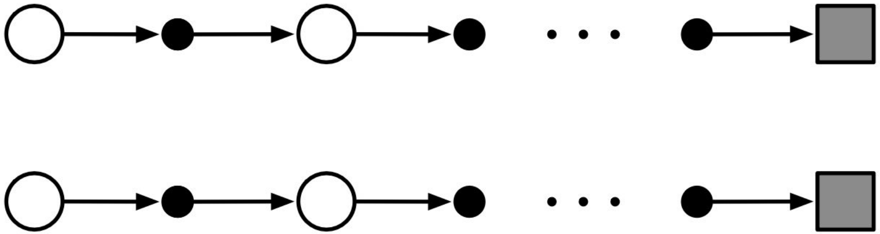
- Only require experience
  - Sample sequences of states, actions, and rewards from actual or
  - Simulated interaction with an environment
- No prior knowledge of the environment's dynamics!
- Yet can still attain optimal behavior
- Main idea: Solving the reinforcement learning problem based on averaging sample returns

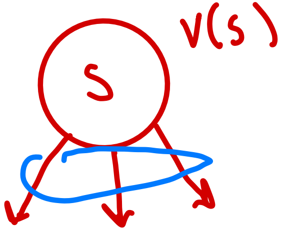
Monte Carlo Methods

- True converge function

- $V_*(s)$

- A true value function: the expected returns of states averaged over <u>all</u> possible episodes
- MC: the expected returns of states averaged over all "sampled" episodes
    - Simply average the returns observed after visits to that state
    - As more returns are observed, the average should converge to the expected value

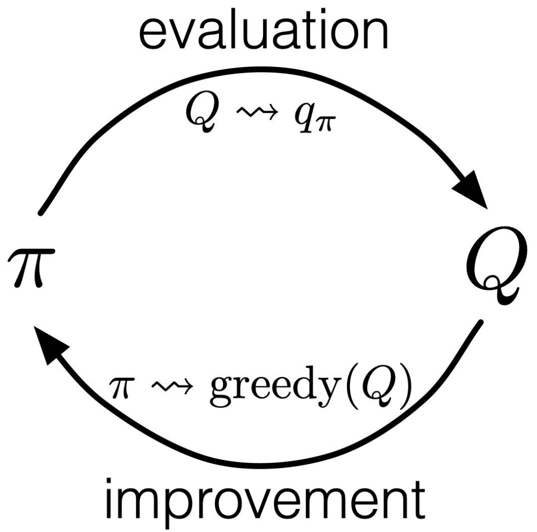




The Prediction and Control Problems

- The prediction problem
  - Estimate the value functions given a policy
- The control problem
  - Approximate optimal policies

Monte-carlo
MC



MC Prediction

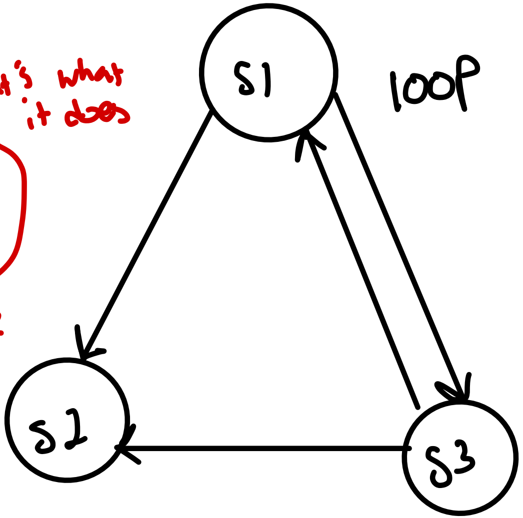
- The same state may appear multiple times in an episode
- We call each appearance a visit to the state
- We can update the value function of the state
    - when it is the first time visit (first-visit MC)
    - Or each time when it's visited (every-visit MC)

that's what it does

v(s1)

disregard this one since duplicate

s1 a r s2 a r s1



MC Prediction

First-visit MC prediction, for estimating $V \approx v_\pi$

Input: a policy π to be evaluated

Initialize:

$V(s) \in \mathbb{R}$, arbitrarily, for all $s \in \mathcal{S}$

$Returns(s) \leftarrow$ an empty list, for all $s \in \mathcal{S}$

starts backwards

Loop forever (for each episode):

Generate an episode following π: $S_0, A_0, R_1, S_1, A_1, R_2, \ldots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$

Loop for each step of episode, $t = T-1, T-2, \ldots, 0$:

$G \leftarrow \gamma G + R_{t+1}$

Unless S_t appears in $S_0, S_1, \ldots, S_{t-1}$:

Append G to $Returns(S_t)$

$V(S_t) \leftarrow \text{average}(Returns(S_t))$

S_1

$[10, 20, 15, \ldots]$

gamma

$G = \text{return} := R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \ldots$

Milestone Questions

SJSU

First-visit MC prediction, for estimating $V \approx v_\pi$

Input: a policy π to be evaluated

Initialize:

$\quad V(s) \in \mathbb{R}$, arbitrarily, for all $s \in \mathcal{S}$

$\quad Returns(s) \leftarrow$ an empty list, for all $s \in \mathcal{S}$

Loop forever (for each episode):

$\quad$ Generate an episode following π: $S_0, A_0, R_1, S_1, A_1, R_2, \ldots, S_{T-1}, A_{T-1}, R_T$

$\quad G \leftarrow 0$

$\quad$ Loop for each step of episode, $t = T-1, T-2, \ldots, 0$:

$\quad\quad G \leftarrow \gamma G + R_{t+1}$

$\quad\quad$ Unless S_t appears in $S_0, S_1, \ldots, S_{t-1}$:

$\quad\quad\quad$ Append G to $Returns(S_t)$

$\quad\quad\quad V(S_t) \leftarrow \text{average}(Returns(S_t))$

- Which part of the pseudo code represents “first-visit”?
- If every-visit, how does it look like?

Milestone Questions

First-visit MC prediction, for estimating $V \approx v_\pi$

Input: a policy π to be evaluated

Initialize:

$\quad V(s) \in \mathbb{R}$, arbitrarily, for all $s \in \mathcal{S}$

$\quad Returns(s) \leftarrow$ an empty list, for all $s \in \mathcal{S}$

Loop forever (for each episode):

$\quad$ Generate an episode following π: $S_0, A_0, R_1, S_1, A_1, R_2, \ldots, S_{T-1}, A_{T-1}, R_T$

$\quad G \leftarrow 0$

$\quad$ Loop for each step of episode, $t = T-1, T-2, \ldots, 0$: alpha is step size

$\quad\quad G \leftarrow \gamma G + R_{t+1}$ [illegible]

$\quad\quad$ Unless S_t appears in $S_0, S_1, \ldots, S_{t-1}$:

$\quad\quad\quad$ ~~Append G to $Returns(S_t)$~~ $V(S_t) \leftarrow V(S_t) + \alpha(G_t - V(S_t))$

$\quad\quad\quad V(S_t) \leftarrow \text{average}(Returns(S_t))$

- Which part of the pseudo code represents “first-visit”?
- If every-visit, how does it look like?

Milestone Questions

- Assume an agent observed two episodes
  - S1 A2 3 S2 A1 4 T
  - S2 A3 1 S3 A1 10 S2 A2 -2 T
- Using first-visit MC Prediction, compute the state value
- For simplicity,
  - Initialize all state values with 0
  - Assume no discounting

first

skip since not first

$v(s_2)$

$[~~4~~, 4]$

$[9, 4]$

$V(S1) = 0$

$4+3$

$V(S2) = \frac{4+9}{2} = 6.5$

$V(S3) = \frac{8}{1} = 8$

$v(s_2) = \frac{4}{1} = 4$

$v(s_1) = \frac{3+4}{1} = 7$

Effectively Computing the Average

- Each time step gives you a sampled return
- You need to compute the average μ_k until the step k
- A naive way:
  - Store all returns $\{R_1, R_2, \ldots, R_k\}$
  - Sum them up
  - Divide the sum by k
- Any problem?

Effectively Computing the Average

- Incremental average
- The mean μ_1, μ_2, ... of a sequence $x_1, x_2, \ldots$ can be computed incrementally

$$
\begin{aligned}
\mu_k &= \frac{1}{k}\sum_{j=1}^{k} x_j \\
&= \frac{1}{k}\left(x_k + \sum_{j=1}^{k-1} x_j\right) \\
&= \frac{1}{k}\left(x_k + (k-1)\mu_{k-1}\right) \\
&= \mu_{k-1} + \frac{1}{k}\left(x_k - \mu_{k-1}\right)
\end{aligned}
$$

Effectively Computing the Average

- Incremental average
- The mean μ_1, μ_2, ... of a sequence $x_1, x_2, \ldots$ can be computed incrementally

$$\begin{aligned}
\mu_k &= \frac{1}{k}\sum_{j=1}^{k} x_j \\
&= \frac{1}{k}\left(x_k + \sum_{j=1}^{k-1} x_j\right) \\
&= \frac{1}{k}\left(x_k + (k-1)\mu_{k-1}\right) \\
&= \mu_{k-1} + \frac{1}{k}\left(x_k - \mu_{k-1}\right)
\end{aligned}$$

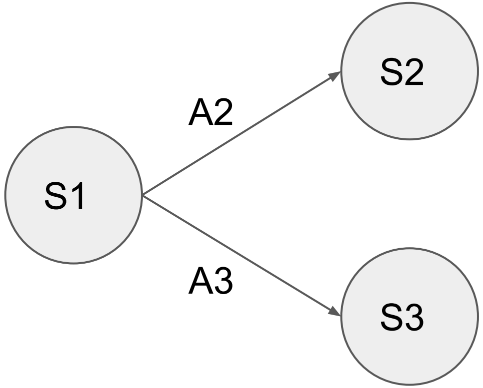
MC Control

- If a model is not available, it is particularly useful to estimate action values rather than state values
- With a model,
  - One simply looks ahead one step and chooses whichever action leads to the best combination of reward and next state
- Without a model,
  - One must explicitly estimate the value of each action
  - Thus, a goal for Monte Carlo methods is to estimate action values

Unless the pair S_t, A_t appears in $S_0, A_0, S_1, A_1 \ldots, S_{t-1}, A_{t-1}$:
  Append G to $Returns(S_t, A_t)$
  $Q(S_t, A_t) \leftarrow \text{average}(Returns(S_t, A_t))$

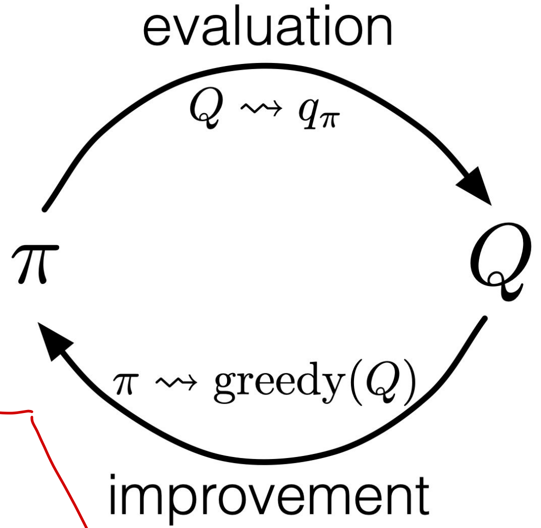
MC Control

- MC Prediction
  - ✓ Estimate the q values by sampling episodes
- MC Control
  - ○ Take a greedily best action from each state to update the policy? NO!



Q(~~S2~~ S1, A2) = 0.2 [after 1st episode]

Q(~~S3~~ S1, A3) = 0 [initial value; never updated]



A Challenge in MC Control

- Many state-action pairs may never be visited
- If the agent follows a deterministic policy, it will observe returns only for one of the actions from each state
- With no returns to average, the Monte Carlo estimates of the other actions will not improve with experience
- The purpose of learning action values is to help in choosing among the actions available in each state
- To compare alternatives we need to estimate the value of all the actions from each state, not just the one we currently favor
- *Maintaining Exploration*

A Soft Policy

- A policy is soft if and only if
  - $\pi(a|s) > 0$ for all state s and all action a
- An agent uses a soft policy to maintain exploration and gradually shifts closer and closer to a deterministic optimal policy
- A very famous soft policy is epsilon-greedy method

Loop forever:

$$A \leftarrow \begin{cases} \arg\max_a Q(a) & \text{with probability } 1-\varepsilon \quad \text{(breaking ties randomly)} \\ \text{a random action} & \text{with probability } \varepsilon \end{cases}$$

Epsilon-greedy Algorithm

epsilon/3
since 3 action

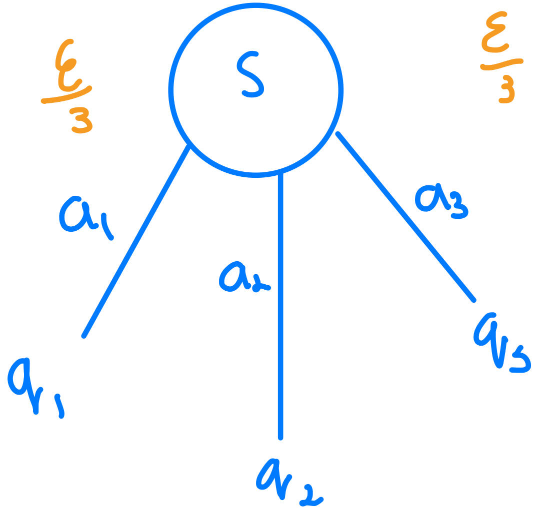
- All nongreedy actions are given the minimal probability of selection

$$\frac{\varepsilon}{|\mathcal{A}(s)|}$$

- The remaining bulk of is given to the greedy action

$$1 - \varepsilon + \frac{\varepsilon}{|\mathcal{A}(s)|}$$

- This is a soft policy when $\varepsilon > 0$
- Also, this is close to the greedy algorithm



Milestone Questions

- From the current state s,
- There are five actions a1, a2, …, a5
- Current estimate of q-values are:
  - Q(s,a1) = 0.2
  - Q(s,a2) = 1.0 Best one
  - Q(s,a3) = 0.8
  - Q(s,a4) = 0.2
  - Q(s,a5) = 0
- Compute the probability that each action will be selected during the next visit to s under the epsilon-greedy algorithm (epsilon = 0.1)

$0.1/5$

$1 - \varepsilon + \frac{\varepsilon}{5} = 92$

$1 - 0.1 + \frac{\varepsilon}{1} = 1$

On-policy first-visit MC control (for ε-soft policies), estimates $\pi \approx \pi_*$

Algorithm parameter: small $\varepsilon > 0$

Initialize:

$\pi \leftarrow$ an arbitrary ε-soft policy

$Q(s,a) \in \mathbb{R}$ (arbitrarily), for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$

$Returns(s,a) \leftarrow$ empty list, for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$

Repeat forever (for each episode):

Generate an episode following π: $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$

Loop for each step of episode, $t = T-1, T-2, \ldots, 0$:

$G \leftarrow \gamma G + R_{t+1}$

Unless the pair S_t, A_t appears in $S_0, A_0, S_1, A_1 \ldots, S_{t-1}, A_{t-1}$:

Append G to $Returns(S_t, A_t)$

$Q(S_t, A_t) \leftarrow \text{average}(Returns(S_t, A_t))$ ← Q values

$A^* \leftarrow \arg\max_a Q(S_t, a)$ (with ties broken arbitrarily)

For all $a \in \mathcal{A}(S_t)$:

$$\pi(a|S_t) \leftarrow \begin{cases} 1 - \varepsilon + \varepsilon/|\mathcal{A}(S_t)| & \text{if } a = A^* \\ \varepsilon/|\mathcal{A}(S_t)| & \text{if } a \neq A^* \end{cases}$$

ε-greedy

prediction

Policy improvement

Keeping a Running Mean of Returns

- MC Prediction update with a learning rate α

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[G - Q(S_t, A_t)]$$

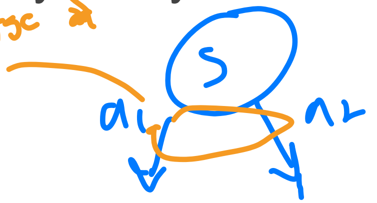
- Forgets older episodes
- Effective for non-stationary problems

Epsilon-greedy Improves the Policy?

go a_1 or a_2

Assume π is any epsilon-soft policy, and π' is the epsilon-greedy policy

average



$$
\begin{aligned}
q_\pi(s, \pi'(s)) &= \sum_a \pi'(a|s) q_\pi(s,a) \\
&= \frac{\varepsilon}{|\mathcal{A}(s)|} \sum_a q_\pi(s,a) \; + \; (1-\varepsilon) \max_a q_\pi(s,a) \\
&\geq \frac{\varepsilon}{|\mathcal{A}(s)|} \sum_a q_\pi(s,a) \; + \; (1-\varepsilon) \sum_a \frac{\pi(a|s) - \frac{\varepsilon}{|\mathcal{A}(s)|}}{1-\varepsilon} q_\pi(s,a)
\end{aligned}
$$

Epsilon-greedy Improves the Policy?

Assume π is any epsilon-soft policy, and π' is the epsilon-greedy policy

$$
\begin{aligned}
q_\pi(s, \pi'(s)) &= \sum_a \pi'(a|s) q_\pi(s,a) \\
&= \frac{\varepsilon}{|\mathcal{A}(s)|} \sum_a q_\pi(s,a) \; + \; (1-\varepsilon) \max_a q_\pi(s,a) \\
&\geq \frac{\varepsilon}{|\mathcal{A}(s)|} \sum_a q_\pi(s,a) \; + \; (1-\varepsilon) \sum_a \frac{\pi(a|s) - \frac{\varepsilon}{|\mathcal{A}(s)|}}{1-\varepsilon} q_\pi(s,a) \\
&= \frac{\varepsilon}{|\mathcal{A}(s)|} \sum_a q_\pi(s,a) \; - \; \frac{\varepsilon}{|\mathcal{A}(s)|} \sum_a q_\pi(s,a) \; + \; \sum_a \pi(a|s) q_\pi(s,a) \\
&= v_\pi(s).
\end{aligned}
$$

By the policy improvement theorem, $\pi' \geq \pi$

The Policy Improvement Theorem

- Let π and π' be any pair of deterministic policies such that, for all states s,

$$q_\pi(s, \pi'(s)) \geq v_\pi(s)$$

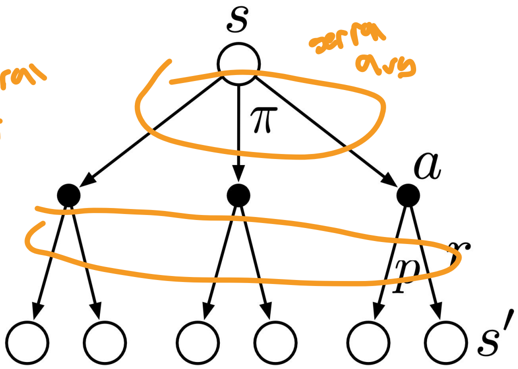
- Then, the policy π' must be as good as, or better than π, for all states s,

$$v_{\pi'}(s) \geq v_\pi(s)$$

general avg

if Q avg ≥ general av

Q avg



Milestone Questions

- There are two actions (Left or Right) from each state
- The current Q table shows
  - Q(S1, L) = 1.0, Q(S1, R) = 0
  - Q(S2, L) = - 1.0, Q(S2, R) = 2.0
  - Q(S3, L) = 5.0, Q(S3, R) = - 1.0
- Assume an agent observed the following episode
  - S1 L 3 S2 R 4 S3 R 10 T
  - This episode was generated, following the epsilon-greedy
- Using On-policy first-visit MC control,
  - Compute the action value (assume no discounting for simplicity)
  - Update the policy function

S3: 10

S2: 4

S1: 17

$10/5 = 2$

0.01

$Q(s1, L) = 1 + \alpha(17 - 1.0)$

$= 1.16$

$Q(S2, R) = 2.0 + \alpha(14 - 2.0)$

$= 2.0 + 0.01(12) = 2.12$

$Q(S3, R) = -1 + 0.01(10 + 1)$

$= -0.89$

On-policy vs. Off-policy

- On-policy
  - Estimate the value of a policy while using it for control
  - i.e., A single policy function to generate episodes and to update
- Off-policy
  - Two separate policies for the episode generation and update
  - Behavior policy: The policy used to generate behavior
  - Target policy: The policy evaluated and improved
- An advantage of this separation is that the target policy may be deterministic (e.g., greedy), while the behavior policy can continue to sample all possible actions

Off-policy MC control, for estimating $\pi \approx \pi_*$

Initialize, for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$:

$Q(s, a) \in \mathbb{R}$ (arbitrarily)

$C(s, a) \leftarrow 0$

$\pi(s) \leftarrow \arg\max_a Q(s, a)$ (with ties broken consistently)

Loop forever (for each episode):

$b \leftarrow$ any soft policy

Generate an episode using b: $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$

$W \leftarrow 1$

Loop for each step of episode, $t = T-1, T-2, \ldots, 0$:

$G \leftarrow \gamma G + R_{t+1}$

$C(S_t, A_t) \leftarrow C(S_t, A_t) + W$

$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \frac{W}{C(S_t, A_t)} [G - Q(S_t, A_t)]$ greedy

$\pi(S_t) \leftarrow \arg\max_a Q(S_t, a)$ (with ties broken consistently)

If $A_t \neq \pi(S_t)$ then exit inner Loop (proceed to next episode)

$W \leftarrow W \frac{1}{b(A_t|S_t)}$

Off-policy MC control, for estimating $\pi \approx \pi_*$

Initialize, for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$:

$Q(s, a) \in \mathbb{R}$ (arbitrarily)

$C(s, a) \leftarrow 0$

$\pi(s) \leftarrow \arg\max_a Q(s, a)$ (with ties broken consistently)

Loop forever (for each episode):

$b \leftarrow$ any soft policy

Generate an episode using b: $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$

$W \leftarrow 1$

Loop for each step of episode, $t = T-1, T-2, \ldots, 0$:

$G \leftarrow \gamma G + R_{t+1}$

$C(S_t, A_t) \leftarrow C(S_t, A_t) + W$

represents difference between b & pi

$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \frac{W}{C(S_t, A_t)}[G - Q(S_t, A_t)]$

$\pi(S_t) \leftarrow \arg\max_a Q(S_t, a)$ (with ties broken consistently)

If $A_t \neq \pi(S_t)$ then exit inner Loop (proceed to next episode)

$W \leftarrow W \frac{1}{b(A_t|S_t)}$

b episode X G_x

$\tilde{\pi}$ never experience X

Importance Sampling

- Can we use any policy as the behavior policy b?
- No!
  - Every action taken under the target policy is also taken, at least occasionally, under b (the assumption of coverage)
- Weighting returns according to the relative probability of their trajectories occurring under the target and behavior policies (the importance-sampling ratio)

Importance Sampling

- Given a starting state St, the probability of the subsequent state–action trajectory

target policy

$$\Pr\{A_t, S_{t+1}, A_{t+1}, \ldots, S_T \mid S_t, A_{t:T-1} \sim \pi\}$$
$$= \pi(A_t|S_t)p(S_{t+1}|S_t, A_t)\pi(A_{t+1}|S_{t+1}) \cdots p(S_T|S_{T-1}, A_{T-1})$$

like sigma but instead multiply

action prob · transition

$$= \prod_{k=t}^{T-1} \pi(A_k|S_k)p(S_{k+1}|S_k, A_k),$$

b

- The importance-sampling ratio is

$$\rho_{t:T-1} \doteq \frac{\prod_{k=t}^{T-1} \pi(A_k|S_k)p(S_{k+1}|S_k, A_k)}{\prod_{k=t}^{T-1} b(A_k|S_k)p(S_{k+1}|S_k, A_k)} = \prod_{k=t}^{T-1} \frac{\pi(A_k|S_k)}{b(A_k|S_k)}.$$

soft policy

Importance Sampling

- Recall that we wish to estimate the expected returns (values) under the target policy
- But all we have are returns G due to the behavior policy
- These returns have the wrong expectation $\boxed{\mathbb{E}[G_t | S_t = s] = v_b(s)}$
- The importance-sampling ratio transforms the returns to have the right expected value

$$\mathbb{E}[\rho_{t:T-1} G_t \mid S_t = s] = v_\pi(s).$$

- A way to implement this expectation is weighted importance sampling

$$V(s) \doteq \frac{\sum_{t \in \mathcal{T}(s)} \rho_{t:T(t)-1} G_t}{\sum_{t \in \mathcal{T}(s)} \rho_{t:T(t)-1}}$$

the set of all time steps in which state s is visited

Importance Sampling

- With all episodes starting from a state s

$$V_n \doteq \frac{\sum_{k=1}^{n-1} W_k G_k}{\sum_{k=1}^{n-1} W_k}, \qquad n \geq 2,$$

- Online update with the cumulative sum of the weights, C

$$C_{n+1} \doteq C_n + W_{n+1},$$

$$V_{n+1} \doteq V_n + \frac{W_n}{C_n}\Big[G_n - V_n\Big], \qquad n \geq 1,$$

Incremental Implementation

Off-policy MC prediction (policy evaluation) for estimating $Q \approx q_\pi$

Input: an arbitrary target policy π
Initialize, for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$:
    $Q(s,a) \in \mathbb{R}$ (arbitrarily)
    $C(s,a) \leftarrow 0$

Loop forever (for each episode):
    $b \leftarrow$ any policy with coverage of π
    Generate an episode following b: $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$
    $G \leftarrow 0$
    $W \leftarrow 1$
    Loop for each step of episode, $t = T-1, T-2, \ldots, 0$, while $W \neq 0$:
        $G \leftarrow \gamma G + R_{t+1}$
        $C(S_t, A_t) \leftarrow C(S_t, A_t) + W$
        $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \frac{W}{C(S_t,A_t)} [G - Q(S_t, A_t)]$
        $W \leftarrow W \frac{\pi(A_t|S_t)}{b(A_t|S_t)}$

Off-policy MC Control

Off-policy MC control, for estimating $\pi \approx \pi_*$

Initialize, for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$:
  $Q(s,a) \in \mathbb{R}$ (arbitrarily)
  $C(s,a) \leftarrow 0$
  $\pi(s) \leftarrow \arg\max_a Q(s,a)$ (with ties broken consistently) Greedy policy (GPI)

Loop forever (for each episode):
  $b \leftarrow$ any soft policy
  Generate an episode using b: $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$
  $G \leftarrow 0$
  $W \leftarrow 1$
  Loop for each step of episode, $t = T-1, T-2, \ldots, 0$:
    $G \leftarrow \gamma G + R_{t+1}$
    $C(S_t, A_t) \leftarrow C(S_t, A_t) + W$
    $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \frac{W}{C(S_t,A_t)} [G - Q(S_t, A_t)]$
    $\pi(S_t) \leftarrow \arg\max_a Q(S_t, a)$ (with ties broken consistently)
    If $A_t \neq \pi(S_t)$ then exit inner Loop (proceed to next episode)
    $W \leftarrow W \frac{1}{b(A_t|S_t)}$